

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

**образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №3

По дисциплине “Информационные технологии и программирование”

Выполнил: студент Петунин Алексей
группы БПИ2401

Проверил: Харрасов Камиль Раисович

Москва

2025

Цель работы:

Изучение и реализация хэш-таблицы с методом цепочек, а также практическое использование класса HashMap.

Задание 1. Реализация HashTable

Структура данных

```
private LinkedList<Entry<K, V>>[] table;  
private int size;  
private int capacity;
```

Entry - класс для хранения пар ключ-значение:

```
public Entry(K key, V value) {  
    this.key = key;  
    this.value = value;  
}
```

Ключевые методы

Хэш-функция:

```
private int hash(K key) {  
    return Math.abs(key.hashCode()) % capacity;  
}
```

Вычисляет индекс багета для ключа

Метод put():

```
public void put(K key, V value) {  
    int index = hash(key);  
  
    if (table[index] == null) {  
        table[index] = new LinkedList<>();  
    }  
  
    for (Entry<K, V> entry : table[index]) {  
        if (entry.getKey().equals(key)) {  
            entry.setValue(value);  
            return;  
        }  
    }  
  
    table[index].add(new Entry<>(key, value));  
    size++;  
}
```

Метод get():

```
public V get(K key) {
    int index = hash(key);

    if (table[index] != null) {
        for (Entry<K, V> entry : table[index]) {
            if (entry.getKey().equals(key)) {
                return entry.getValue();
            }
        }
    }
    return null;
}
```

Рехэширование

При заполнении более 75% таблица увеличивается в 2 раза:

```
if ((double) size / capacity > LOAD_FACTOR) {
    rehash();
}
```

Задание 2. Работа с HashMap

Класс Student

```
static class Student {
    private final String firstName;
    private final String lastName;
    private final int age;
    private double averageGrade;

    public Student(String firstName, String lastName, int age, double averageGrade) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.averageGrade = averageGrade;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;

        Student student = (Student) o;
        return age == student.age &&
            Double.compare(student.averageGrade, averageGrade) == 0 &&
            Objects.equals(firstName, student.firstName) &&
            Objects.equals(lastName, student.lastName);
    }

    @Override
    public int hashCode() {
        return Objects.hash(firstName, lastName, age, averageGrade);
    }

    @Override
    public String toString() {
        return String.format("%s %s, возраст: %d, средний балл: %.2f",
            firstName, lastName, age, averageGrade);
    }
}
```

equals() сравнивает все поля объекта, включая средний балл

hashCode() использует те же поля, что и equals(), обеспечивая согласованность

toString() предоставляет читаемое строковое представление студента

Результаты тестирования

```
1. Добавление студентов:
=== Содержимое хэш-таблицы ===
Размер: 3, Емкость: 16
Бакет 0: пусто
Бакет 1: пусто
Бакет 2: пусто
Бакет 3: пусто
Бакет 4: пусто
Бакет 5: пусто
Бакет 6: пусто
Бакет 7: пусто
Бакет 8: пусто
Бакет 9: пусто
Бакет 10: пусто
Бакет 11: пусто
Бакет 12: [2023003=Мария Сидорова, возраст: 19, средний балл: 4,80]
Бакет 13: [2023002=Петр Петров, возраст: 21, средний балл: 4,20]
Бакет 14: [2023001=Иван Иванов, возраст: 20, средний балл: 4,50]
Бакет 15: пусто
=====
2. Поиск студента:
Найден: Петр Петров, возраст: 21, средний балл: 4,20
3. Обновление студента:
Обновленный: Иван Иванов, возраст: 21, средний балл: 4,70
4. Удаление студента:
Удален: Мария Сидорова, возраст: 19, средний балл: 4,80
Размер после удаления: 2
5. Проверка методов:
Размер таблицы: 2
Таблица пуста: false
Содержит ключ 2023001: true
6. Демонстрация коллизий:
=== Содержимое хэш-таблицы ===
Размер: 3, Емкость: 5
```

```
Бакет 0: пусто
Бакет 1: [1=Значение 1] [6=Значение 6] [11=Значение 11]
Бакет 2: пусто
Бакет 3: пусто
Бакет 4: пусто
=====
7. Демонстрация рехэширования:
Рехэширование выполнено. Новая емкость: 8
Рехэширование выполнено. Новая емкость: 16
=== Содержимое хэш-таблицы ===
Размер: 10, Емкость: 16
Бакет 0: пусто
Бакет 1: [key0=0]
Бакет 2: [key1=1]
Бакет 3: [key2=2]
Бакет 4: [key3=3]
Бакет 5: [key4=4]
Бакет 6: [key5=5]
Бакет 7: [key6=6]
Бакет 9: [key8=8]
Бакет 10: [key9=9]
Бакет 11: пусто
Бакет 12: пусто
Бакет 13: пусто
Бакет 14: пусто
Бакет 10: [key9=9]
Бакет 11: пусто
Бакет 12: пусто
Бакет 13: пусто
Бакет 14: пусто
Бакет 12: пусто
Бакет 13: пусто
Бакет 14: пусто
Бакет 14: пусто
Бакет 15: пусто
```

Контрольные вопросы:

1. Для чего нужен класс Object?

Базовый класс всех Java-объектов, предоставляет основные методы: equals(), hashCode(), toString().

2. Для чего нужно переопределять методы equals() и hashCode()?

Для корректной работы с коллекциями (HashMap, HashSet). Если объекты равны по equals(), их hashCode() должен совпадать.

3. Какие есть правила переопределения методов equals() и hashCode()?

Если equals() возвращает true, hashCode() должен быть одинаковым

Если equals() возвращает false, hashCode() может совпадать (коллизия)

Методы должны быть согласованы и использовать одни и те же поля

4. Что делает метод toString()? Почему его часто переопределяют?

Возвращает строковое представление объекта. Переопределяют для удобства отладки и читаемого вывода.

5. Что делает метод finalize()? Почему его использование считается устаревшим?

Вызывается перед удалением объекта сборщиком мусора. Устарел из-за непредсказуемости вызова.

6. Что такое коллизия?

Ситуация, когда разные ключи имеют одинаковый хэш-код.

7. Какие есть способы разрешения коллизий?

Метод цепочек (chaining) - элементы с одинаковым хэшем хранятся в списке

Открытая адресация - поиск следующей свободной ячейки

8. Как хранятся данные в хэш-таблице?

В массиве бакетов, где каждый бакет может быть пустым или содержать список элементов.

9. Что происходит, если в хэш-таблицу добавить элемент с одинаковым значением ключа?

Значение перезаписывается, размер таблицы не изменяется.

10. Что происходит, если в хэш-таблицу добавить элемент с таким же хэш-кодом ключа, но разными исходными значениями?

Происходит коллизия - оба элемента хранятся в одном бакете как разные записи.

11. Как изменяется HashMap при достижении порогового значения?

Происходит рехэширование - размер таблицы увеличивается в 2 раза, все элементы перераспределяются.